

P1212R0

Modules and Freestanding

Published Proposal, 2018-10-06

This version:

<https://wg21.link/P1212R0>

Author:

[Ben Craig](#) (National Instruments)

Audience:

EWG, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ben-craig/freestanding_proposal/blob/master/core/freestanding_modules.bs

Abstract

Standard library modules needs to know a direction for freestanding.

Table of Contents

1 Introduction

2 Potential directions

- 2.1 Continue with the status quo (Not recommended)
- 2.2 Eliminate freestanding (Not recommended)
- 2.3 Expand the library, maintain the language (Weakly recommend)
- 2.4 Expand the library, subset the language (RECOMMENDED)

3 Related Polls

- 3.1 SG14 cppcon meeting polls for D1105R1.3 (Sep 26, 2018)

- 3.2 SG14 telecon polls for P1105R0 (July 11, 2018)
- 3.3 LEWG poll for P0829R0 (Nov 8, 2017)
- 3.4 LEWG poll for P0581R1 (Mar 13, 2018)

References

Informative References

§ 1. Introduction

[\[P0581\]](#) proposes a modularization of the standard library. The paper calls out "Robust support for C++ in diverse environments" as a goal.

[\[P0829\]](#) proposes many additions to the freestanding subset of C++. The subset is attempting to be the maximal subset of the library with no OS requirements or space overhead. It could also be considered a lowest common denominator subset.

[\[P1105\]](#) proposes removing core features and the closely related parts of the library that don't work in kernel and embedded environments. It attempts to make a lowest common denominator subset of the language.

WG21 needs to get some early consensus on where we want to take freestanding C++. There are several options. This choice will likely affect the partitioning of the standard library modules.

§ 2. Potential directions

§ 2.1. Continue with the status quo (Not recommended)

C++17 and the current working draft's freestanding has the core language as *_almost_* identical to the hosted core language. [intro.multithread](#) and [basic.start.main](#) are the only language differences. The library is substantially reduced. Most of the facilities in the freestanding standard library are there to support core language features, and are facilities the user could not portably write themselves. There are a few "mundane" facilities (like `<bit>`) that are in the freestanding library that users could write, and there are some core language features that can't be accessed with the freestanding library (like `<string_view>`, and by extension, `<contract>`).

I believe that the current split is neither consistent nor useful in practice.

§ 2.2. Eliminate freestanding (Not recommended)

Allegedly, some portion of WG21 is against sub-setting the language in general, despite the fact that we have a subset already in place. Since the current freestanding subset isn't useful, I find the idea of removing freestanding as self-consistent and reasonable, even though I do not personally agree with it. This would leave us with one language.

This option would require the least amount of work from WG21.

SG14 polls were strongly against this option.

§ 2.3. Expand the library, maintain the language (Weakly recommend)

We could choose to leave the core language as is for freestanding (or even unify it by removing the [intro.multithread](#) and [basic.start.main](#) wording), but address freestanding purely at the library level.

We could add facilities to the freestanding library that don't require an OS, and generally work on kernel and embedded systems. This is the approach [\[P0829\]](#) takes with the current headers. This approach could be replicated for standard library modules.

The difficulty of stopping here is that it requires us to engage in some double-think. On the one hand, we know that exceptions (and other core language features) aren't suitable on many systems, so we avoid facilities in the library that require those features. On the other hand, we would be keeping those facilities in the language, despite knowing they aren't suitable. Kernel and embedded systems would still need to resort to non-standard C++ for the bulk of their code.

If we take this approach, then we would end up putting a few operations into the freestanding library that don't make much sense on kernel and embedded systems. In particular, we would likely end up with the `std::exit` family of functions, and exception management functions like `std::uncaught_exceptions`.

LEWG has polled as in favor of this approach as of Albuquerque 2017. The "subset the language" option did not have a paper at the time.

§ 2.4. Expand the library, subset the language (RECOMMENDED)

We could subset the language, so that the new subset truly is a zero-overhead language, where you only pay for the features you are using. Further, we could make the new subset only contain features that are available everywhere. The big features that would need to be cut would be exceptions, RTTI, thread-local storage, and the default heap. For a more in-depth list, including rationale, see [\[P1105\]](#).

This option would steer us in the general direction of [\[P0829\]](#) and [\[P1105\]](#). The standard library modules would be split accordingly, so that some set of modules could always be available, no matter the platform.

I feel that this approach is acknowledging existing practice, and would be a boon to C++ in the kernel and embedded spaces.

SG14 polls were strongly in favor of this approach.

§ 3. Related Polls

§ 3.1. SG14 cppcon meeting polls for D1105R1.3 (Sep 26, 2018)

Minutes

Poll: I want to know if we're on board with a way to disable dynamic, type-based exceptions (this proposal is neutral with respect to static exceptions)
(no opposition in this room)

§ 3.2. SG14 telecon polls for P1105R0 (July 11, 2018)

Minutes

Poll 1: get rid of freestanding

SF/F/N/A/SA

0/1/2/9/11

Poll 2: modify freestanding along the lines of [\[P1105\]](#), encouragement for further work, agree with most of it

SF/F/N/A/SA

5/13/4/0/0

§ 3.3. LEWG poll for P0829R0 (Nov 8, 2017)

Minutes

Change the definition of freestanding along these lines

SF/F/N/A/SA

1/7/7/1/0

Add a classification for embedded-friendly interfaces

SF/F/N/A/SA

1/5/9/1/0

We support proposed removal of typeinfo and exception headers from freestanding

SF/F/N/A/SA

1/6/6/1/2

We are ok with marking freestanding support on the per-API level, not per-header

SF/F/N/A/SA

0/15/0/3/0

We are ok with marking freestanding support on a method level, not per-class.

SF/F/N/A/SA

0/9/3/5/1

§ 3.4. LEWG poll for P0581R1 (Mar 13, 2018)

Minutes

We want to see more work on defining module(s) specifically targeting basic/freestanding.

UNANIMOUS CONSENT

§ References

§ Informative References

[P0581]

Marshall Clow; et al. [Standard Library Modules](http://wg21.link/P0581). URL: <http://wg21.link/P0581>

[P0829]

Ben Craig. [Freestanding Proposal](http://wg21.link/P0829). URL: <http://wg21.link/P0829>

[P1105]

Ben Craig; Ben Saks. [Leaving no room for a lower-level language: A C++ Subset](http://wg21.link/P1105). URL: <http://wg21.link/P1105>